TP2 / PRÁCTICA DE LABORATORIO

Control supervisor de un servomotor industrial y análisis de DYNAMIXEL Protocol 2.0

ESP32-S3 N16R8 · RS-485 · ROBOTIS H42-20-S300-R · Interfaz Flask

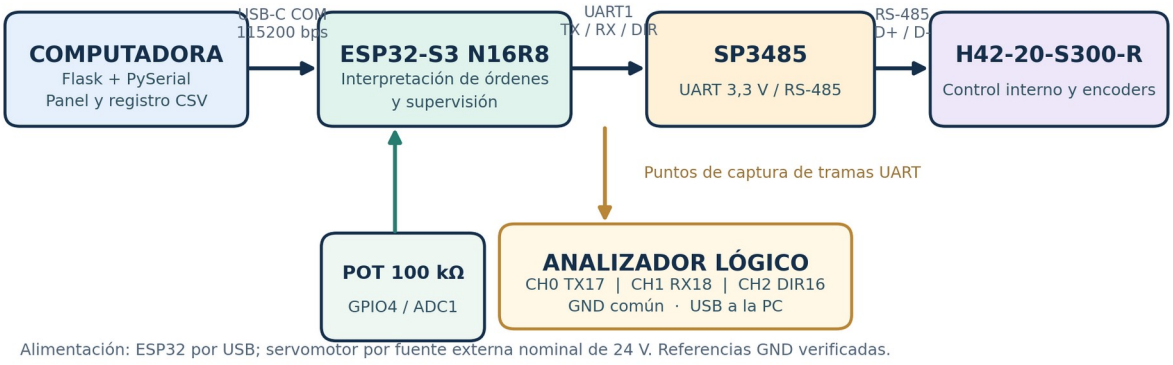


Figura 1. Arquitectura de la práctica: comando, transmisión, realimentación y adquisición de datos.

Documento técnico de referencia para montaje, ejecución de ensayos, análisis de tramas y registro de resultados.

Alcance: firmware Control_H42_PC_Arduino.ino y aplicación Panel_Dynamixel_Flask. Las mediciones presentadas como ejemplos no representan ensayos realizados sobre el equipo.

Versión de referencia · Noviembre de 2026

1. Introducción y alcance

Los servomotores industriales integran accionamiento, sensores y controladores electrónicos capaces de ejecutar consignas de posición, velocidad y torque. En esta práctica se emplea un ROBOTIS DYNAMIXEL PRO H42-20-S300-R, comandado por una ESP32-S3 a través de un transceptor SP3485 y un bus RS-485 semidúplex. La computadora actúa como estación de operación: transmite consignas por USB y recibe datos del actuador para visualizarlos en un panel web local desarrollado con Python y Flask.

El propósito principal no es construir desde cero un controlador PID externo. El DYNAMIXEL ya implementa lazos internos de posición y velocidad. La tarea de la ESP32 es supervisar el conjunto: validar parámetros, limitar las consignas solicitadas y traducir órdenes de alto nivel a instrucciones DYNAMIXEL Protocol 2.0.

1.1 Objetivos

- Identificar las funciones y conexiones de la ESP32-S3, el transceptor y el servo.
- Explicar la diferencia entre UART, RS-485 y DYNAMIXEL Protocol 2.0.
- Reconocer y decodificar paquetes Instruction y Status, campos ID, LENGTH, Instruction, Parameters y CRC.
- Aplicar límites de posición, velocidad y aceleración mediante comandos enviados desde la PC.
- Comparar consigna, posición medida y variables operativas mediante la interfaz Flask.
- Capturar el tráfico UART con el analizador lógico y vincular cada paquete con la operación ejecutada.

1.2 Alcance y versiones

La implementación documentada usa el firmware para el modelo H42-20-S300-R original (número de modelo 51200). La variante H42-20-S300-R(A) tiene un mapa de registros distinto y no debe utilizarse con las direcciones aquí indicadas sin adaptar el programa. La información se basa en los archivos de firmware y Flask revisados, y en la documentación técnica oficial ROBOTIS.

2. Componentes del montaje

Componente	Función en el sistema
PC con Windows y USB	Carga del sketch, ejecución del panel Flask y almacenamiento de CSV.
ESP32-S3 N16R8	UART1 hacia el transceptor, puerto serie USB a PC, supervisión y ADC del potenciómetro.
SP3485, 3,3 V, semidúplex	Convierte UART de niveles lógicos a señales diferenciales RS-485.
ROBOTIS H42-20-S300-R	Actuador inteligente con encoder y control interno; nominal 24 V.
Fuente de 24 V	Potencia del servo; independiente del USB de la ESP32.
Potenciómetro 100 kΩ (opcional)	Consigna analógica alternativa en MODE POT, conectada a GPIO4.
Analizador lógico 24 MHz, 8 CH	Captura en paralelo de TX, RX y dirección del bus, desde el lado UART.
Software	Arduino IDE + Dynamixel2Arduino; Python + Flask + PySerial y navegador web.

3. Esquema de conexión y cableado

En el montaje se distinguen tres dominios: alimentación de potencia del servo (24 V), lógica y UART de la ESP32 (3,3 V), y transmisión diferencial RS-485. Estos niveles no deben confundirse.

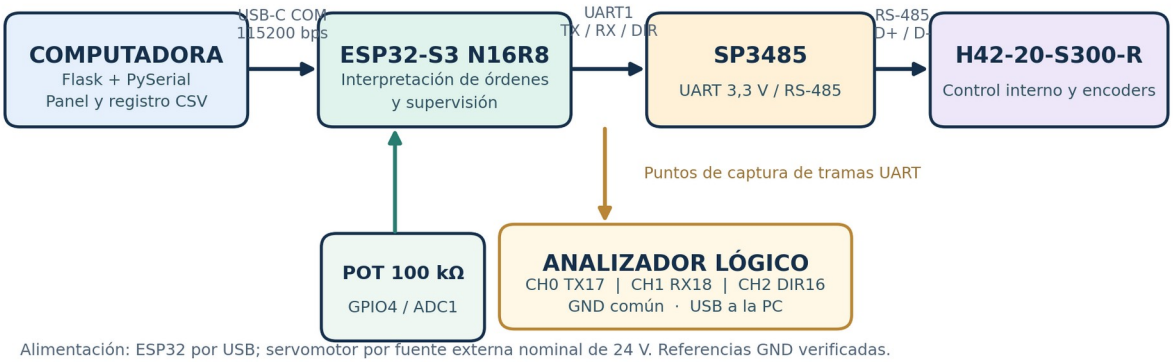


Figura 2. Vista general de la conexión y puntos accesibles de medición.

3.1 ESP32 ↔ módulo SP3485

ESP32-S3	Módulo Candy-H / SP3485	Función
3V3	VCC	Alimentación del transceptor (3,3 V).
GND	GND	Referencia eléctrica del circuito no aislado.
GPIO17 (UART1 TX)	DI	Datos enviados al bus.
GPIO18 (UART1 RX)	RO	Datos recibidos del bus.
GPIO16	RSE o DE + /RE	Control de sentido en semidúplex.

El módulo utilizado en el montaje presenta un pin combinado RSE. En transceptores con DE y /RE separados, ambos pueden compartir la señal de dirección si se respeta su polaridad. GPIO16 en estado alto habilita la transmisión y en bajo habilita la recepción, según la lógica del circuito indicado. Verificar la serigrafía real de cada módulo.

3.2 Conexión entre RS-485 y servo

SP3485	Servo / conector ROBOTIS	Comprobación
A (D+)	Pin 3 DATA+	Comprobar rotulado A/B del módulo.
B (D-)	Pin 4 DATA-	Comprobar polaridad de las líneas.
Referencia GND	Pin 1 GND de comunicación	Referencia común cuando el montaje no es aislado.
No conectar alimentación del servo a la ESP32	VDD, pin 2 del conector de comunicación; alimentación por conector de potencia dedicado	La fuente nominal del servo es 24 V.

NOTA TÉCNICA ROBOTIS indica expresamente el uso del conector dedicado de 24 V para energizar la serie PRO. Confirmar la orientación de pines por numeración del conector; el color del cable no determina su función.

3.3 Potenciómetro y analizador lógico

Elemento	Conexión recomendada	Finalidad
Potenciómetro (extremo 1)	ESP32 3V3	Referencia superior del divisor.
Potenciómetro (cursor)	GPIO4 (ADC1)	Lectura de posición analógica.
Potenciómetro (extremo 3)	GND	Referencia inferior del divisor.
Saleae CH0	GPIO17 / DI	Paquetes enviados desde ESP32.
Saleae CH1	GPIO18 / RO	Paquetes de respuesta del servo.
Saleae CH2	GPIO16 / RSE	Sentido de transmisión/recepción.
Saleae GND	GND de ESP32	Referencia de los niveles lógicos.

Para el potenciómetro de 100 kΩ puede añadirse un capacitor de 100 nF del cursor a GND para estabilizar el ADC. El analizador debe conectarse en paralelo, sin interrumpir el circuito, y solo sobre señales dentro de los límites eléctricos de su modelo. Los clones “24 MHz / 8 CH” no deben suponerse aptos para conectarse directamente a A/B, ni a 24 V.

4. Interfaz física RS-485

RS-485 es una especificación de transmisión diferencial: un transceptor cambia la relación de potencial entre dos conductores y el receptor interpreta la diferencia Vdiff = V(DATA+) - V(DATA-). No transporta, por sí mismo, un significado de comando. Los bytes y la interpretación de sus campos pertenecen al protocolo DYNAMIXEL.

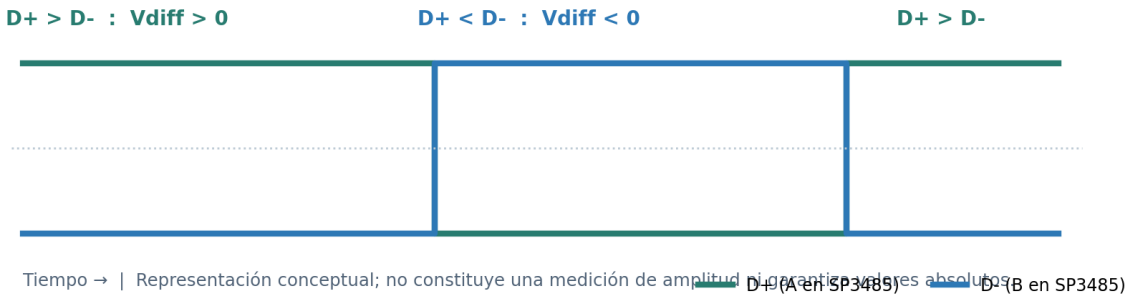


Figura 3. Concepto de señales complementarias y diferencia de potencial. Valores de tensión no medidos.

4.1 Comunicación semidúplex

1. Transmisión: la ESP32 activa GPIO16/RSE, envía los bytes por TX17; el SP3485 convierte los niveles UART al par diferencial.
2. Cambio de sentido: finalizada la transmisión, la línea de habilitación cambia y el transceptor pasa a escuchar al bus.
3. Recepción: el servo responde con otro paquete; el SP3485 entrega los bits recibidos al RX18 de la ESP32.

La versión del firmware examinada utiliza UART1 a 57600 bps, 8 bits de datos, sin paridad y 1 bit de parada (8N1), mientras que el puerto serie PC–ESP32 utiliza 115200 bps. Son dos enlaces diferentes, con baudios independientes. Una configuración previa del servo puede haber alterado el ID o su velocidad de bus.

4.2 Recomendaciones de montaje

- Utilizar par trenzado para DATA+/DATA− y cables lo más cortos posible en el banco.
- Comprobar masa de referencia, continuidad, polaridad A/B y calidad de empalmes antes de energizar.
- Verificar las resistencias de terminación presentes en ambos extremos antes de añadir una de 120 Ω; evitar duplicaciones.
- Alimentar el servo desde una fuente de 24 V con corriente suficiente y mantener físicamente accesible el corte de potencia.

5. DYNAMIXEL Protocol 2.0

DYNAMIXEL Protocol 2.0 define cómo identificar a cada actuador y cómo escribir o leer sus registros. La ESP32 envía un Instruction Packet; el servo devuelve un Status Packet si corresponde según la instrucción y su configuración de respuesta. Cada trama incorpora CRC-16 para verificar la integridad de los datos. El protocolo utiliza transferencia de bytes con orden LSB primero para los valores multibyte.



Figura 4. Campos del paquete de instrucciones y ejemplo PING al ID 1.

5.1 Campos de la trama

Campo	Descripción
FF FF FD / 00	Cabecera de tres bytes y byte reservado.
ID	Dirección del dispositivo: 0–252; 254 = broadcast.
Length L, H	Longitud desde Instruction hasta CRC incluido (sin contar la cabecera/ID/length).
Instruction	01 PING; 02 READ; 03 WRITE; 55 identifica un Status Packet.
Parameters	Datos específicos: dirección del registro, cantidad de bytes o valor escrito.
CRC L, H	CRC-16 para comprobar la integridad del paquete.

Además del CRC, el protocolo define byte stuffing: si la secuencia FF FF FD aparece dentro del cuerpo del paquete, se inserta un byte FD para evitar que se interprete como una nueva cabecera. La biblioteca Dynamixel2Arduino realiza automáticamente el ensamblado, stuffing, cálculo del CRC y decodificación de respuestas.

5.2 Ejemplos para capturar con el Saleae

PING ID=1: FF FF FD 00 01 03 00 01 19 4E

READ posición 611: FF FF FD 00 01 07 00 02 63 02 04 00 1B F9

WRITE 0° a 596: FF FF FD 00 01 09 00 03 54 02 00 00 00 00 33 09

Los tres paquetes corresponden a ejemplos teóricos válidos para Protocol 2.0 e ID 1; no son capturas de laboratorio. En READ, la dirección 611 se codifica 63 02 (little-endian) y se solicitan cuatro bytes. En WRITE, Goal Position se encuentra en la dirección 596 (54 02), seguida de cuatro bytes de datos. El paquete Status utiliza Instruction = 55, incluye un campo Error y, en las lecturas, los parámetros solicitados.

6. Mapa de registros utilizado por el firmware

Registro (dirección)	Uso	Unidad o codificación
Torque Enable (562)	Habilita / libera la retención del eje	0 / 1
Goal Position (596)	Consigna de posición	303750 pulsos / vuelta
Goal Velocity (600)	Límite de velocidad en modo posición	0,00329218 rpm / raw
Goal Acceleration (606)	Perfil de aceleración	58.000 rpm² del motor interno / raw
Moving (610)	Movimiento o perfil en ejecución	0 / 1
Present Position (611)	Posición real	303750 pulsos / vuelta
Present Velocity (615)	Velocidad del eje	0,00329218 rpm / raw
Present Current (621)	Corriente de bobina	4,02832 mA / raw
Present Input Voltage (623)	Tensión interna de alimentación	0,1 V / raw
Present Temperature (625)	Temperatura medida	1 °C / raw
Hardware Error Status (892)	Indicadores de falla de hardware	Bits / hexadecimal

El firmware también consulta: Operating Mode (11), Status Return Level (891), Velocity Limit (32), Acceleration Limit (26), Max Position Limit (36) y Min Position Limit (40). Las direcciones de estas variables corresponden al H42 original, no a la versión (A).

6.1 Escalas y ejemplos de conversión

Posición: pulsos = grados × 303750 / 360. Por ejemplo, +50° equivale aproximadamente a +42188 pulsos. Velocidad: rpm de salida = raw × 0,00329218. Por ejemplo, VEL 5 rpm equivale a unas 1519 unidades RAW de Goal Velocity (según el redondeo del firmware). Corriente: mA = raw × 4,02832; en el sketch se interpreta el registro como entero de 16 bits con signo.

El servo original admite un recorrido nominal de -180° a +180° en modo posición, sujeto a sus límites internos, al offset y a la instalación mecánica. Que la especificación mecánica indique ±180° no autoriza automáticamente mover el brazo a esos extremos.

7. Arquitectura de control y niveles de limitación

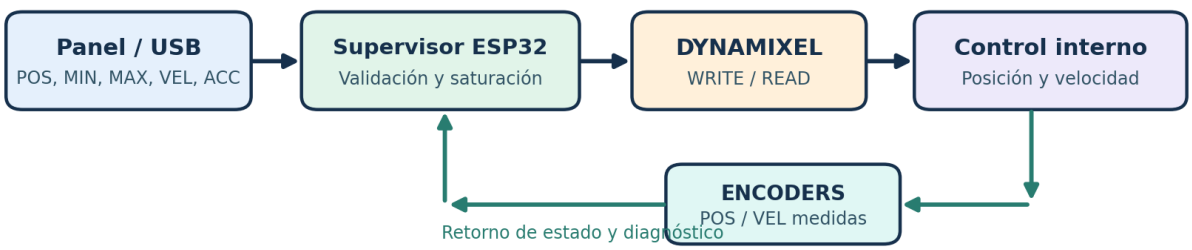


Figura 5. La ESP32 supervisa consignas; el controlador de lazo cerrado reside dentro del servomotor.

El control de la práctica se organiza en tres capas: (1) la interfaz gráfica valida y envía comandos textuales; (2) el firmware de la ESP32 verifica el estado y satura posiciones; (3) el H42 ejecuta el control interno con sus sensores y valores de configuración. Los límites de software no reemplazan límites mecánicos ni una función de seguridad certificada.

7.1 Restricciones aplicadas

Nivel	Ejemplo en el proyecto	Consecuencia
Interfaz Flask	POS/MIN/MAX admitidos dentro de ±180°; VEL y ACC poseen validaciones de entrada del panel.	La interfaz rechaza comandos mal formados, pero no define el rango real.
Límites de banco en ESP32	BANCO_MIN_GRADOS = -50; BANCO_MAX_GRADOS = +50; BANCO_MAX_RPM = 20; BANCO_MAX_ACC_RAW = 25.	No permite ampliar por comandos más allá de estos topes sin recompilar.
Límites del servo (EEPROM)	Min/Max Position Limit, Velocity Limit, Acceleration Limit.	Son leídos al iniciar. El firmware respeta su intersección con el banco.
Mecánica y seguridad	Topes, interferencias con el brazo, sujeción y corte de potencia.	Pueden imponer restricciones adicionales no detectadas por software.

NOTA TÉCNICA MIN y MAX modifican únicamente límites supervisores en RAM de la ESP32. No reescriben EEPROM ni garantizan detener inmediatamente una trayectoria ya ordenada si el nuevo límite se reduce durante el movimiento. Los cambios deben realizarse con el equipo detenido y verificados antes de ensayar.

7.2 Consignas y control interno

El algoritmo convierte el ángulo en un Goal Position y lo limita al intervalo permitido. VEL ajusta Goal Velocity; ACC modifica Goal Acceleration. En modo posición, el controlador interno regula el desplazamiento con realimentación. La lectura de Present Position permite comparar el objetivo y la posición real; el error e(t) = objetivo(t) − posición_real(t) puede analizarse para medir respuesta temporal. El firmware no modifica las ganancias PID internas del H42.

7.3 Comportamiento de ARM, OFF y STOP

- ARM lee la posición real, configura velocidad/aceleración, escribe esa misma posición como consigna y habilita torque. El arranque del sketch no ordena movimiento automáticamente.
- STOP lee la posición actual y la utiliza como nueva consigna, además de pasar al modo PC. Es una detención controlada; no es parada de emergencia.
- OFF solicita Torque Enable = 0. El eje deja de sostener activamente la posición; no usar con cargas suspendidas.

8. Programa de la ESP32: operación y comandos

Archivo de referencia: Control_H42_PC_Arduino.ino. La ESP32 recibe comandos ASCII de una línea por el puerto USB-C COM; cada comando se termina con "Nueva línea". El sketch utiliza la biblioteca Dynamixel2Arduino para leer y escribir registros de la tabla de control.

Comando	Operación	Observación
PING	Identifica o recupera la comunicación.	No permitido durante movimiento o después

Comando	Operación	Observación
		de un fallo bloqueante.
INFO	Imprime modo, límites software/EEPROM y registros Goal.	Diagnóstico de configuración.
STATUS	Fuerza una lectura de las ocho variables.	La telemetría puede estar incompleta si alguna consulta falla.
ARM / OFF	Habilita / deshabilita torque.	ARM inicializa consigna desde posición medida.
MODE PC / MODE POT	Alterna control por terminal o por ADC.	En POT se usa GPIO4 y rampa de 30°/s.
POS <grados>	Solicita una posición, aplicando saturación.	Solo disponible en MODE PC y armado.
MIN / MAX <grados>	Modifica límites software.	Deben caber en el rango del banco y EEPROM.
VEL <rpm>	Configura límite de velocidad.	0,5…20 rpm en el firmware de referencia.
ACC <RAW>	Configura aceleración.	Entero de 1 a 25 en el firmware de referencia.
STOP	Mantiene la posición real actual.	No equivale a parada de emergencia.
STREAM ON/OFF	Activa u oculta la impresión periódica.	Las consultas internas continúan para supervisión.
RATE <ms>	Modifica período de lectura/reporte.	100…5000 ms; predeterminado 500 ms.
HELP	Imprime ayuda desde la ESP32.	Sin movimiento.

8.1 Línea de telemetría

t=10420 | MODE=PC | MIN=-30.0 | MAX=40.0 | OBJ=40.00 | POS=32.15deg | VEL=8.72rpm | I=210.3mA | VIN=24.1V | T=32C | MOV=1 | TORQ=1 | HW=0x00

Los valores anteriores son ilustrativos. t es millis() desde el arranque de la ESP32; OBJ corresponde a la última consigna aplicada, POS/VEL/I/VIN/T a registros del servo; MOV indica movimiento/perfil, TORQ indica torque habilitado y HW resume errores. Las lecturas se realizan mediante varias consultas sucesivas, por lo que no son muestras eléctricamente simultáneas.

8.2 Supervisión de fallas

El programa detiene las órdenes nuevas y solicita torque OFF ante tres ciclos consecutivos de telemetría incompleta, un Hardware Error Status no nulo o la pérdida inesperada del estado TORQ. Esto reduce el riesgo, pero la supervisión depende de la comunicación y no sustituye el corte físico de potencia.

9. Interfaz gráfica Python + Flask

La aplicación Panel_Dynamixel_Flask abre el puerto serie COM mediante PySerial. Flask sirve una interfaz web local en http://127.0.0.1:5000. Los botones y el cuadro de texto envían los mismos comandos aceptados por la ESP32; el navegador no controla el servo directamente ni implementa DYNAMIXEL.



Figura 6. Distribución funcional de controles y gráficos según la interfaz Flask (sin datos simulados).

9.1 Módulos y flujo de datos

Módulo	Responsabilidad
app.py	Servidor Flask, conexión/desconexión COM, lectura serie, validación de comandos, API y CSV.
telemetry.py	Parsea líneas t=..., variables de estado y mensajes INFO.
templates/index.html	Interfaz: selección de COM, control, diagnósticos y telemetría.
static/app.js y app.css	Gráficos locales, actualización del estado y estilos.
requirements.txt	Dependencias de Python.
INICIAR_PANEL.bat	Inicio asistido en Windows.

El programa conserva hasta 1200 muestras en memoria durante la conexión y permite exportarlas a CSV. Los indicadores se actualizan únicamente si la ESP32 transmite líneas válidas; el panel distingue puerto serie conectado de telemetría reciente. Los gráficos son de posición objetivo/real y velocidad/corriente; también se visualizan tensión, temperatura y estados.

9.2 Instalación en PC

4. Instalar Python 3.10 o superior y descomprimir el paquete Panel_Dynamixel_Flask.
5. Abrir PowerShell dentro de la carpeta del proyecto y ejecutar: py -m pip install -r requirements.txt
6. Cerrar el Monitor Serie de Arduino: el puerto COM solo puede estar abierto por una aplicación a la vez.
7. Ejecutar py app.py (alternativamente INICIAR_PANEL.bat) y abrir http://127.0.0.1:5000.
8. Seleccionar COM6 o el puerto detectado, presionar Conectar, enviar INFO y STREAM ON.
9. Desde el panel, probar comandos de diagnóstico antes de habilitar ARM y después exportar el CSV.

NOTA TÉCNICA El control desde el navegador depende de que Flask tenga acceso exclusivo al COM. El panel puede admitir valores numéricos más amplios que el firmware; la validación definitiva corresponde a la ESP32 y a los límites del servo.

10. Protocolo experimental de laboratorio

Las siguientes pruebas deben realizarse con el motor descargado o con una articulación asegurada, sin recorrido obstruido, con corte físico de potencia accesible. Registrar primero la configuración leída por INFO y no confundir los ejemplos con resultados reales.

10.1 Ensayo A: identificación y comunicación

```
PING
INFO
STATUS
STREAM ON
RATE 500
```

Criterio: detectar modelo 51200, obtener variables válidas y visualizar líneas de telemetría. En el analizador se espera un Instruction Packet PING, un Status Packet y varias operaciones READ.

10.2 Ensayo B: control angular y saturación

```
MODE PC
VEL 5
ACC 5
ARM
POS 0
MIN -30
MAX 40
POS 60
POS -50
```

Registrar en la terminal Pedido, Aplicado y SATURADO. Los últimos dos comandos deberían aplicarse como +40° y −30° si los límites del banco y la EEPROM lo permiten. Esperar a que cada trayectoria termine antes de enviar la siguiente. No cambiar límites mientras el servo esté en movimiento.

10.3 Ensayo C: comparación de velocidades

```
POS -30
VEL 5
POS 40
# Esperar fin del movimiento; volver a -30
POS -30
VEL 15
POS 40
```

Comparar el mismo recorrido con 5 rpm y 15 rpm. Extraer del CSV tiempo t, OBJ, POS y VEL para comparar tiempo total, máximo de velocidad y evolución de posición. Los comandos precedidos con # son indicaciones para la persona que ensaya, no instrucciones para enviar literalmente.

10.4 Ensayo D: perfil de aceleración

```
VEL 15
ACC 2
POS -30
POS 40
# Repetir desde -30 después de llegar
ACC 15
POS -30
POS 40
```

Mantener la misma distancia y velocidad límite, cambiando la aceleración. Evaluar la evolución de POS y VEL. La interpretación debe tener en cuenta la dinámica del servo y la frecuencia de muestreo de la aplicación.

10.5 Ensayo E: respuesta a un escalón

```
RATE 100
STREAM ON
POS 0
# Comprobar posición estable y MOV=0
POS 40
```

Exportar CSV, graficar θ_objetivo(t) y θ_real(t), estimar tiempo de establecimiento y error final. Para comparar ensayos, usar la misma velocidad y aceleración y condiciones iniciales equivalentes.

10.6 Prueba de seguridad y cierre

```
STOP
STATUS
OFF
```

Verificar el significado de STOP y que OFF libera el eje. Si existe carga suspendida, evitar OFF hasta asegurar mecánicamente la articulación. Desenergizar la fuente del servo al finalizar.

11. Uso del analizador lógico en el TP

Conectar CH0 a TX17, CH1 a RX18, CH2 a DIR16 y GND a masa común. Capturar al menos un PING, una lectura de Present Position y una escritura de Goal Position. Añadir dos decodificadores Async Serial, 8N1, sin inversión, y el baudrate efectivo del bus (p. ej. 57600). El canal DIR permite distinguir qué extremo está autorizado a transmitir.

Acción	Instrucción que debe identificarse	Registro o campo relevante
PING	01 (PING)	ID, modelo, versión de firmware en Status.
POS 40	03 (WRITE)	Goal Position, dirección 596 (54 02).
VEL 5	03 (WRITE)	Goal Velocity, dirección 600 (58 02).
ACC 5	03 (WRITE)	Goal Acceleration, dirección 606 (5E 02).
STATUS / stream	02 (READ)	Present Position 611, velocidad 615, corriente 621, etc.

No se debe interpretar el canal USB entre la PC y la ESP32 como una trama DYNAMIXEL: allí circulan comandos de texto ASCII y telemetría formateada. El protocolo DYNAMIXEL está en el enlace UART1–SP3485–RS-485–servo. El analizador lógico genérico se conecta al lado UART de 3,3 V, no directamente al cableado diferencial sin una interfaz de medición apta.

12. Planilla de resultados

Ensayo	Parámetros enviados	Observación / archivo CSV / captura
A. PING y lectura	ID / baud: _____	_____
B. Saturación	MIN: _____ MAX: _____	Pedido: _____ Aplicado: _____
C. Velocidad	VEL: 5 y 15 rpm	Tiempo 1: _____ Tiempo 2: _____
D. Aceleración	ACC: 2 y 15 RAW	Perfil y diferencias: _____
E. Escalón	0° a +40°	ts: _____ s; error final: _____°

13. Conclusiones técnicas previstas

La práctica permite observar el funcionamiento de un actuador inteligente mediante un protocolo digital sobre RS-485. El aporte principal de la ESP32 es la validación y supervisión de consignas, mientras que el posicionamiento real depende de los lazos internos del DYNAMIXEL. La interfaz Flask agrega visualización de telemetría, experimentación reproducible y exportación de datos. El Saleae permite relacionar cada comando con la trama transmitida y con la respuesta del servo.

Los resultados reales deberán incorporarse luego de los ensayos. El sistema no implementa parada de emergencia certificada ni reemplaza verificaciones eléctricas y mecánicas propias del laboratorio.

14. Referencias

- ROBOTIS. DYNAMIXEL PRO H42-20-S300-R. https://emanual.robotis.com/docs/en/dxl/pro/h42-20-s300-r/
- ROBOTIS. DYNAMIXEL Protocol 2.0. https://emanual.robotis.com/docs/en/dxl/protocol2/
- ROBOTIS. Dynamixel2Arduino. https://github.com/ROBOTIS-GIT/Dynamixel2Arduino
- Espressif Systems. ESP32-S3 UART / RS-485. https://docs.espressif.com/projects/esp-idf/en/stable/esp32s3/api-reference/peripherals/uart.html
- Proyecto de firmware: Control_H42_PC_Arduino.ino (versión de referencia).
- Proyecto de interfaz: Panel_Dynamixel_Flask (app.py, telemetry.py, templates/index.html y static).
- UNLaM, Electrónica Industrial. Material de cátedra: Servomotores y Clase 10 – Sensores industriales.